

오늘의 작업기록 — 2026년 7월 7일

ledger-api — POST /transfer: mini-ledger의 Bank가 진짜 API가 된 날 (m105 완주)

(도메인은 한 줄도 안 바꿨다 — 호출자만 Main에서 curl로 갈아끼운 날)

한눈에

항목	내용
목표	m105 조각 2 — Bank를 HTTP에 태워 POST /transfer, 잔액 변경 확인
결과	m105 완주 ✓ — 멍등성이 HTTP에서 실전 작동, update.py --done m105
커밋	5d4d4de (server.log 정리) → 이식/record → d0d6f9c "add POST /transfer wired to mini-ledger Bank (verified: idempotent over HTTP)"
오늘의 그림	5단계 중 ④호출 한 칸만 교체: Main.java → curl→컨트롤러. 도메인(검증·이체·수수료·기록·멍등성·복식부기) 무변
★ 처음 만난 개념	record(DTO) · @PostMapping/@RequestBody(JSON 자동 변환 = 마법 ④) · 상태를 가진 컨트롤러(요청 간 Bank 유지) · curl POST 플래그 · git rm --cached · sed '1i'
검증 3연타	key-1 이체됨 → key-1 재시도 잔액 불변(멍등성) → key-2 정상 처리 + "돈 보존됨" ×2
트러블 3건	IntelliJ 멍통→VPS nano 전환 · 404=파일 부재(nano 저장 실패) · 서버 내림 판정=ss 빈 출력
다음	예외→HTTP 상태코드 번역(잔액 부족이 500으로 새는 것) · 복기 예약(원자성 트레이스+checkBalance)

어제 띄운 서버는 인사(/health)만 했다. 오늘은 진짜 일을 시켰다 — 6/19부터 쌓은 Bank(원자성 검증·멍등성·복식부기까지 든 그 원장)를 그대로 이식해, JSON 요청 한 방으로 이체가 처리되게 했다. 핵심은 도메인을 한 글자도 안 고쳤다는 것. Bank를 깔때기로 만들어둔 설계가 오늘 값을 했다 — 호출자만 갈아끼우면 되는 구조였다. 그 리고 6/29에 만든 멍등성이 오늘 Stripe·토스가 쓰는 바로 그 API 패턴으로 실전 검증됐다.

0. 오늘의 그림 — 바뀐 것은 딱 한 칸

[요청 만들기] → [값 전달] → [도메인 실행] → ④[호출자] → [결과 확인]

지금까지: ④ = Main.java가 bank.transfer("a","b",1000,"key-1") 호출

오늘부터: ④ = curl(JSON) → 스프링 → TransferController → bank.transfer(...)

- 도메인 실행부(Bank·Transaction·예외들)는 오늘 수정 0줄. 이식하며 package 한 줄씩 엮은 게 전부다.
- Main.java는 안 가져왔다 — Main의 역할(호출·시나리오)을 이제 curl이 대신하기 때문. mini-ledger 본진은 졸업작으로 보존.

PART 1 — ★ 오늘 처음 만난 개념들 (풀코드 + 정밀 해설)

오늘은 mini-ledger(순수 자바)에서 스프링으로 넘어가는 전환기라 처음 보는 게 많았다. 이 파트가 오늘 기록의 본체다.

1-1. record — 자바가 "그릇 클래스"를 한 줄로 만들어주는 문법

작성한 풀코드 (TransferRequest.java 전체):

```
package com.ledger.api;

public record TransferRequest(String fromId, String toId, long amount, String idempotencyKey) {}
```

이게 뭔가: record 는 "데이터를 담기만 하는 클래스"를 선언 한 줄로 만들어주는 자바 문법이다. 위 한 줄에서 자바가 자동 생성해주는 것:

- private final 필드 4개 (fromId, told, amount, idempotencyKey)
- 필드 4개를 전부 받는 생성자
- 각 필드의 접근자(꺼내는 메서드)
- equals/hashCode/toString (덤)

내가 이미 손으로 만들어본 것과의 대조 — Transaction 클래스를 짤 때 나는 이 패턴을 전부 수동으로 쳤다: private final long id; 같은 필드 선언들, 그걸 다 받는 생성자, getId() 같은 게터들. 약 30줄. record는 그 30줄을 괄호 안 선언으로 압축한 것이다. 즉 record는 새로운 개념이 아니라 내가 이미 아는 패턴(불변 데이터 그릇)의 자동화다.

★ 함정 — 접근자 이름이 다르다:

```
request.getFromId() // ✕ record에는 이런 메서드가 없다
request.fromId()    // ✓ get 없이 필드명 그대로 + 괄호
```

게터 관례(getXxx)에 익숙해진 손이 그대로 나가면 컴파일 에러다. record는 필드명 자체가 접근자 이름이다.

왜 여기 들어갔나: curl이 보내는 JSON(`{ "fromId": "a", ... }`)을 받아줄 **타입 있는 그릇**이 필요했다. 이런 "요청/응답 데이터 전용 그릇"을 실무에서 **DTO**(Data Transfer Object)라 부른다. 불변인 것도 맞다 — 요청 데이터가 처리 도중 바뀌면 안 되니까. Transaction을 불변으로 만든 것과 같은 철학이다.

1-2. @RequestBody + 잭슨 — JSON이 record가 되는 마법 ④

컨트롤러 시그니처:

```
@PostMapping("/transfer")
public String transfer(@RequestBody TransferRequest request) { ... }
```

이게 뭔가: `@RequestBody` 는 "HTTP 요청의 본문(body)을 이 매개변수 타입으로 변환해서 꽂아라"는 스티커다. 실제 일어나는 흐름:

```
curl이 보낸 것: { "fromId": "a", "toId": "b", "amount": 1000, "idempotencyKey": "key-1" }  (그냥 텍스트)
↓ ① 스프링이 Content-Type: application/json 헤더를 보고 "본문이 JSON이구나"
↓ ② 잭슨(Jackson, 스프링에 내장된 JSON 변환기)이 텍스트를 파싱
↓ ③ JSON의 키 이름(fromId)과 record의 필드명(fromId)을 짝지어
↓ ④ new TransferRequest("a", "b", 1000, "key-1")를 대신 호출
request 매개변수에 완성된 객체가 꽂힘
```

★ **핵심 규칙:** JSON 키 이름 = record 필드명. 이 이름 일치로 매핑된다. `fromId` 를 JSON에서 `from_id` 로 보내면 못 찾는다.

왜 마법 목록에 넣나: 내가 파싱 코드를 한 줄도 안 썼는데 텍스트가 객체가 됐다. 스프링이 대신해주는 것 목록에 ④ **JSON↔객체 자동 변환**으로 추가한다. (기존: ①Tomcat 기동 ②컴포넌트 스캔 ③URL→메서드 연결.)

1-3. @PostMapping + curl POST — GET과 뭐가 다른가

- **GET** (어제 /health): "조회". 본문 없음. `curl URL` 만 치면 기본이 GET.
- **POST** (오늘 /transfer): "생성/변경". **본문에 데이터를 실어 보낸다.** 이체는 서버 상태(잔액)를 바꾸는 요청이니 POST가 맞다.

오늘의 curl — 플래그 3개가 처음:

```
curl -X POST http://localhost:8080/transfer \
-H "Content-Type: application/json" \
-d '{"fromId": "a", "toId": "b", "amount": 1000, "idempotencyKey": "key-1"}'
```

- `-X POST` = 요청 방식 지정 (안 쓰면 GET)
- `-H "Content-Type: application/json"` = "본문이 JSON"이라는 **라벨**. 이게 없으면 스프링/잭슨이 본문을 JSON으로 해석하지 않는다 (1-2의 ①단계가 이 헤더를 본다)
- `-d '...'` = 본문(body) 자체

1-4. ★ 상태를 가진 컨트롤러 — 오늘의 개념 핵심

작성한 풀코드 (`TransferController.java` 전체, 커밋 d0d6f9c):

```
package com.ledger.api;

import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class TransferController {

    private final Bank bank = new Bank();

    public TransferController() {
        bank.createAccount("a", "이름", 2100);
        bank.createAccount("b", "이름", 3500);
        bank.createAccount("platform", "플랫폼", 0);
    }

    @PostMapping("/transfer")
    public String transfer(@RequestBody TransferRequest request) {
        bank.transfer(request.fromId(), request.toId(), request.amount(), request.idempotencyKey());
        return "a: " + bank.getAccount("a").getBalance()
            + ", b: " + bank.getAccount("b").getBalance()
            + ", platform: " + bank.getAccount("platform").getBalance();
    }
}
```

줄 단위로 — 뭐가 왜 들어갔나:

- `private final Bank bank = new Bank();` — Bank를 필드로 둔다. 컨트롤러 객체는 서버가 뜰 때 스프링이 **딱 한 번** 만들고, 서버가 사는 동안 유지한다. 따라서 이 bank도 **한 번 만들어져 계속 산다**. 요청이 100번 와도 **같은 bank** — 잔액·history·usedKeys가 요청들 사이에 이어진다.
- **Main 시절과의 결정적 차이:** `java Main` 은 실행마다 새 Bank였다(끝나면 소멸). 서버 시대엔 **상태가 요청을 넘어 지속된다**. 오늘 검증에서 key-1 2차 요청이 막힌 것이 바로 이 증거다 — 1차 때 저장된 usedKeys가 2차 요청 시점에도 **살아있었기** 때문에 막을 수 있었다.
- **한계(인지만):** 이 상태는 메모리뿐이다. **서버를 재시작하면 잔액·기록 전부 증발**. 이걸 "영속성이 없다"고 한다. 푸는 도구가 나중의 DB/JPA고, 8월 Layer 0의 WAL이 정확히 이 문제(꺾다 쳐도 남는 기록)를 다룬다.
- **생성자의 createAccount 3개** — 서버 부팅 시 1회 초기화. a는 이체+수수료(1020)를 감당하게 2100. **platform 포함** — transfer가 내부에서 `getAccount("platform")` 을 부르므로(수수료 입금), 빼먹으면 첫 이체가 AccountNotFoundException으로 터진다(7/5에 만난 함정, 오늘은 회피).
- **메서드 몸통 2줄** — ① request에서 4개 값을 record 접근자로 꺼내 `bank.transfer(...)` 호출(도메인 실행). ② 세 계좌 잔액을 문자열로 조합해 **return = HTTP 응답 본문**. m105 완료 조건이 "잔액 변경 확인"이라 응답에 잔액이 보이게 설계했다. mini-ledger Main에서 println 하던 그 조합을 return으로 옮긴 것.

1-5. 도구 개념 돌 (git rm --cached, sed '1i')

`git rm --cached server.log` — .gitignore는 "아직 추적 안 된" 파일에만 효과가 있다. server.log는 어제 이미 커밋돼 추적 중이라 ignore에 넣어도 계속 따라온다. `--cached` 는 "추적 목록에서만 제거"(디스크의 파일은 남김) — 그 뒤부터 ignore가 먹는다. 이미 커밋된 파일을 빼는 표준 절차.

`sed -i '1i package com.ledger.api;'` 파일 — "1번 줄 **앞에**(i=insert) 이 텍스트를 삽입." package 선언은 파일 최상단이어야 하므로 '1i'. 여섯 파일에 for 루프로 일괄 적용 — 어제 배운 규칙(package=폴더 경로와 일치)의 자동화.

PART 2 — 작업 흐름 (유의미한 명령 + 판정)

Step 0 — server.log 추적 제거

```
cd ~/ledger-api && echo "server.log" >> .gitignore
git rm --cached server.log && git commit -am "ignore server.log"
# 판정: [main 5d4d4de] ignore server.log
```

Step 1 — 도메인 이식 (6파일 + package 일괄)

```
cd ~/mini-ledger && git pull
cp Account.java Bank.java Transaction.java TransactionType.java \
  AccountNotFoundException.java InsufficientBalanceException.java \
  ~/ledger-api/src/main/java/com/ledger/api/

cd ~/ledger-api/src/main/java/com/ledger/api
for f in Account Bank Transaction TransactionType AccountNotFoundException InsufficientBalanceException; do
  sed -i '1i package com.ledger.api;' "$f.java"
done
head -2 *.java # 판정: 전 파일 첫 줄 = package com.ledger.api;
cd ~/ledger-api && ./gradlew compileJava
# 판정: BUILD SUCCESSFUL in 36s ← 이식 성공 (끊는 지점 A)
```

Step 2 — TransferRequest record (플코드는 1-1)

```
# 작성 후:
./gradlew compileJava # 판정: BUILD SUCCESSFUL in 7s
```

Step 3 — TransferController (플코드는 1-4)

솔로 작성. 골격(임포트·@RestController·Bank 필드·계좌 셋·@PostMapping 시그니처)은 첫 시도에 전부 맞았고, 몸통 2곳을 교정받아 완성 — ① `bank.transfer` 에 인자 없이 멈춰 있던 것 → record 접근자 4개로 채움 ② 빈 return → 잔액 조합으로. 교정의 본질: **mini-ledger에선 값("a", 1000)을 직접 썼지만, 이제 그 값이 request 안에 있으니 꺼내야 한다** — 그게 전부다. 스프링이 바꾼 건 "값이 JSON으로 들어온다"는 것뿐, 그 안은 아는 자바였다.

```
./gradlew compileJava # 판정: BUILD SUCCESSFUL in 8s
```

Step 4 — 기동 → 검증 (PART 4) → 종료

Step 5 — 검증된 것만 커밋 + 마일스톤

```
git add . && git commit -m "add POST /transfer wired to mini-ledger Bank (verified: idempotent over HTTP)" && git push
# 판정: [main d0d6f9c] ... create mode ... TransferController.java / cc07b06..d0d6f9c main -> main

cd ~/backend-journey && python3 update.py --done m105
# 판정: ✓ done 처리: m105 → 푸시 완료
grep -A6 '"id": "m105"' roadmap.json | grep status
# 판정: "status": "done"
```

★ m105 done은 **sed가 아니라 update.py --done** — roadmap.json은 cron의 update.py가 1분마다 재생성하는 파일이라 손 sed는 다음 틱에 덮인다(7/6에 실제로 당한 것). 자동 writer가 있는 파일은 그 인터페이스로만 쓴다. 실행 후 grep으로 done 눈 판정까지가 절차.

PART 3 — 트러블슈팅 3건

3-A. IntelliJ 먹통 → 오늘은 VPS nano로 (우회 판단)

증상: 사지방 PC에서 ledger-api를 clone(File → New → Project from Version Control)하자 Gradle sync가 돌며 **IntelliJ만 완전 먹통**(커서·크롬은 정상). 원인: IntelliJ+Gradle sync(의존성 다운로드+인덱싱)가 이 PC 사양엔 과부하. mini-ledger(순수 자바)는 가벼워 괜찮았지만 스프링 프로젝트는 무겁다 — 어제 bootRun 16분과 같은 뿌리. 판단: 오늘 파일(TransferController)은 짧으니 **VPS nano로 편집** — 어차피 실행(bootRun/curl)이 VPS라 편집·실행을 한곳에서 하는 게 마찰이 적다. **IntelliJ를 버리는 게 아니라 일회성 우회**이고, PC 성능 튜닝은 별도 과제로 미룬다(진도와 안 섞음). Shallow clone 옵션은 체크 해제가 맞다(작은 repo라 아낄 게 없고 히스토리를 잃음).

3-B. 404 Not Found — 진짜 원인은 "파일이 없었다"

증상: 서버 정상 기동 후 POST /transfer → `{"status":404,"error":"Not Found","path":"/transfer"}`. 추적: ls로 폴더 확인 → **TransferController.java가 목록에 없음** → find로 확정(TransferRequest만 존재). **nano에서 짤 게 저장되지 않았던 것**(Ctrl+O 누락 추정). 여기서 배운 것 둘: 1. 404의 뜻 = "그 경로가 서버에 등록돼 있지 않다." 컨트롤러 파일이 없으니 /transfer라는 URL 자체가 존재하지 않았다. 2. **compileJava의 SUCCESSFUL은 "있는 파일들"의 성공이지, "있어야 할 파일"의 존재 증명이 아니다**. 빌드 성공을 보고 파일이 있다고 가정했다 — 판정 순서는 파일 존재(ls/grep)부터. 해결: EOF heredoc으로 재작성(저장 단계가 없어 이런 실수가 원천 차단됨) → `grep -c "PostMapping\|transfer\|RestController" = 6` 판정 → 서버 재기동 → 200.

3-C. 서버 내림의 판정은 kill이 아니라 "ss 빈 출력"

증상: `kill %2` 직후 ss에 java(pid 1178826)가 여전히 LISTEN. gradle(부모)만 죽고 자식 java가 남은 듯 보였다. 전개: `kill 1178826` → "No such process" — 이미 종료돼 있었다(부모 종료가 시차를 두고 자식까지 정리). ss 재확인 → **빈 출력** = 포트 닫힘 확정. 교훈: 공개 IP 서버의 "내렸다" 판정 기준은 kill 명령의 실행이 아니라 `ss -tlnp | grep 8080`의 빈 출력이다. gradle→java 부모·자식 구조 때문에 kill 한 방으로 안 죽을 수도 있으니, 매번 ss로 확인한다.

PART 4 — 검증: m105 완료 조건 (먹등성이 HTTP에서 사는 순간)

```
./gradlew bootRun > server.log 2>&1 &
grep "Started LedgerApi" server.log # 판정: Started ... in 4.702 seconds
```

3연타 curl과 잔액 변화:

#	요청	응답(잔액)	의미
1	key-1 이체 1000	a: 1080, b: 4500, platform: 20	이체 1000+수수료 20이 HTTP로 처리됨
2	key-1 그대로 재전송	a: 1080, b: 4500, platform: 20 (불변)	★ 멍등성이 HTTP에서 작동 — usedKeys가 막음
3	key-2 이체 1000	a: 60, b: 5500, platform: 40	새 키는 정상 처리(같은 키"만" 막는다는 증거)

돈 보존 산수: a 총 -2040(1020×2) = b +2000 + platform +40. 두 이체 모두 온전.

복식부기가 HTTP 뒤에서도 도는가:

```
grep "돈 보존" ~/ledger-api/server.log
# 판정: 돈 보존됨 (합 = 0) ×2 ← 정확히 두 줄
```

"2"라는 숫자 자체가 구조 증명이다 — 실제 이체된 1·3번만 checkBalance(합=0)까지 갔고, 멍등성으로 막힌 2번은 transfer 초입에서 return돼 검증까지 가지 않았다. 어제까지 만든 원자성·복식부기가 HTTP 층 아래에서 그대로 살아 돈다.

이게 왜 큰가: idempotencyKey 로 중복 요청을 흡수하는 이 구조는 Stripe·토스 결제 API의 실제 패턴 그대로다. 네트워크에서 요청은 중복 전송될 수 있고(재시도·타임아웃), 그때 이체가 두 번 되면 사고다. 6/29에 개념으로 만든 것이 오늘 실전 API 계약이 됐다.

(보너스 관찰 예약: 지금 a=60이라 다음 1000 이체는 잔액 부족 — 시도하면 자바 예외가 500으로 샌다. "자바 예외를 HTTP 상태코드로 어떻게 번역하나"가 다음 조각.)

5. 산출물

- 커밋: 5d4d4de (ignore) → 이식/record → d0d6f9c (POST /transfer, verified) — repo Taewoo-Won/ledger-api
- 신규 파일: TransferRequest.java , TransferController.java (폴코드 PART 1)
- 마일스톤: m105 done (update.py --done, grep 판정 완료)
- ledger-api 현재 모습: GET /health + POST /transfer, 뒤에 mini-ledger 도메인 전체(원자성·멍등성·복식부기 포함)

6. 로드맵 위치와 다음

- m105 완주. p1(Foundations)에서 m103(mini-ledger)·m105(첫 Spring API) 완료 — 7월 관문(m107) 사실상 조기 달성 상태.
- 다음 후보: ① 예외→HTTP 번역(잔액 부족=400대, 지금은 500으로 샘) ② 복기 예약(원자성 트레이스+checkBalance Map 빈화면 재현 — 7/8~9, "Map이 아직 손에 안 붙었다") ③ m106 CodeCrafters Redis(7월 후반 슬롯)
- 8월 예고: 오늘 "재시작하면 잔액 증발"을 눈으로 인지했다 — 연속성 부재. Layer 0의 WAL이 정확히 이 문제를 푼다.

7. 검증 질문 (다음 복기용 — 답 적지 말 것)

- record 한 줄이 자동 생성해주는 것 넷은? 접근자 이름 규칙은 게터와 뭐가 다른가?
- @RequestBody가 하는 일은? JSON이 record가 되기까지의 4단계와, 매핑의 핵심 규칙(무엇과 무엇이 일치)은?
- curl POST의 플래그 쉼(-X, -H, -d)은 각각 뭘 지정하나? -H가 없으면 왜 변환이 안 되나?
- Bank를 필드로 둔 것의 의미는 — key-1 2차 요청이 막힐 수 있었던 이유와 연결해 설명하라. 이 상태의 한계는?
- "돈 보존됨"이 정확히 2번인 것이 증명하는 구조는?
- 404의 뜻은? 오늘 404의 진짜 원인과, "compileJava SUCCESSFUL"이 보증하지 않는 것은?
- 공개 IP 서버 "내렸다"의 판정 기준은? kill %N이 한 방에 안 먹을 수 있는 이유는?
- git rm --cached가 필요한 상황은? (.gitignore만으로 안 되는 이유)
- m105 done을 sed가 아니라 update.py --done으로 한 이유는?

8. 회고

오늘 가장 크게 남은 건 두 장면이다.

첫째, 도메인을 한 글자도 안 고치고 API가 됐다. cp로 여섯 파일을 옮기고 package 한 줄씩 엮은 게 전부다. 6/19부터 Bank에 쌓은 검증·멍등성·복식부기가, 호출자를 Main에서 curl로 바꾸는 것만으로 HTTP 세계에서 그대로 돌아왔다. "로직을 Bank라는 깔때기에 모아두면 입구는 갈아끼울 수 있다"는 걸 말로만 알다가 오늘 몸으로 확인했다. 컨트롤러를 짜다 막힌 지점도 사실 스프링이 아니었다 — 값을 직접 쓰던 걸 request에서 꺼낸다"는 한 뼘 차이였고, 그걸 넘으니 나머지는 아는 자바였다. "하나도 모르겠다"고 느꼈지만 골격은 첫 시도에 다 맞았다. 낯설과 모름은 다르다는 것도 오늘의 수확이다.

둘째, key-1을 두 번째 보냈을 때 잔액이 안 움직인 순간. 그 한 줄이 오늘의 하이라이트다. 서버 안의 Bank가 요청 사이에 살아있어서, 1차 때 저장한 usedKeys가 2차를 막았다 — "상태를 가진 서버"라는 개념이 숫자 불변으로 눈앞에 증명됐다. 그리고 이게 Stripe·토스가 쓰는 바로 그 패턴이라는 것. 6/29에 Set 하나로 만든 멍등성이 오늘 실전 API 계약이 됐다. "돈 보존됨"이 정확히 2번 찍힌 것까지 — 막힌 요청은 검증까지 안 간다는 구조가 로그 줄 수로 드러난 게 묘하게 통쾌했다.

삽질도 셋 있었고 셋 다 판정 습관이 잡았다. IntelliJ가 멍통일 때 씨름 대신 nano로 우회한 것(진도가 우선, 성능 튜닝은 별도 과제), 404에서 "빌드 성공 = 파일 존재"라는 가정 이 깨진 것(ls부터가 순서였다), kill 후 ss로 재확인해 부모·자식 프로세스의 시차를 본 것. 이들 연속 같은 문장이 값을 한다 — 가정 말고 확인.

한계도 눈에 넣어줬다. 지금 잔액은 메모리에만 있다. 서버를 꺾다 켜면 a의 60원도, usedKeys도 전부 증발한다. 오늘은 그게 괜찮은 학습 단계지만, 진짜 원장은 꺾다 켜도 남아야 한다 — 8월 WAL이 정확히 그 문제고, 오늘 이 "증발"을 직접 인지한 게 그때의 동기가 될 것이다.

한 줄: 도메인 한 줄 안 바꾸고 호출자만 갈아끼워 Bank를 HTTP API로 — 같은 키 두 번에 잔액이 안 움직이는 순간, 6/29의 역등성이 Stripe·토스의 실전 패턴이 됐다. m105 완주.